

Java 18 — Simple Web Server

Java Version: 18

Year: 2022

Feature: Simple Web Server

UniqueClassName: SimpleWebServer

Definition

Simple Web Server was introduced in **Java 18** through the `jwebserver` command.

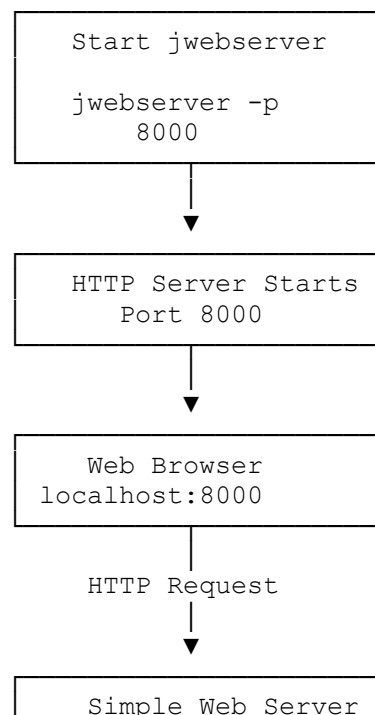
It provides a minimal, command-line HTTP server for serving **static files** from a directory. It is mainly useful for:

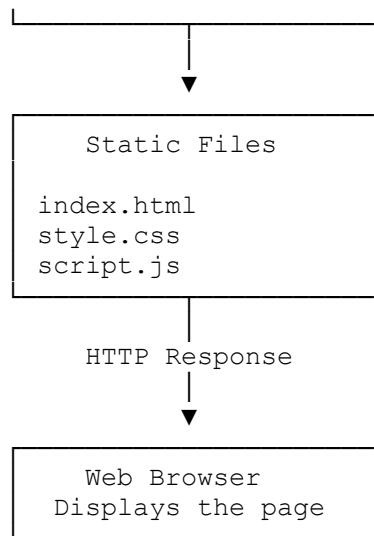
- Testing static HTML/CSS/JavaScript files
- Local development
- Quick file sharing on a development machine
- Learning and experimenting with HTTP

It is **not intended to be a full production web server**.

The underlying API is available through the `jdk.httpserver` module, which provides the lightweight HTTP server functionality.

Flowchart





Java Code Example

Java 18's simple web server is primarily accessed through the **jwebserver command**, rather than by writing a Java server class.

First, create a directory:

```
my-web/  
└─ index.html
```

Create index.html:

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Java 18 Web Server</title>  
</head>  
<body>  
  <h1>Hello from Java 18!</h1>  
  <p>This page is served by the Simple Web Server.</p>  
</body>  
</html>
```

Open a terminal inside the my-web directory and run:

```
jwebserver
```

The default server starts on:

```
http://localhost:8000/
```

Then open that address in your browser.

Specify a Port

You can specify a different port:

```
jwebserver -p 8080
```

Now the server is available at:

```
http://localhost:8080/
```

Specify the Directory

You can also specify the directory containing the files:

```
jwebserver -d C:\projects\my-web
```

For example:

```
jwebserver -d C:\projects\my-web -p 8080
```

The server will expose the files inside that directory.

Using Java's HTTP Server API

Java 18 also provides the underlying lightweight HTTP server API through the `jdk.httpserver` module.

For example:

```
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class SimpleWebServer {

    public static void main(String[] args) throws IOException {

        // Create HTTP server on port 8000
        HttpServer server =
            HttpServer.create(
```

```

        new InetSocketAddress(8000),
        0
    );

    // Create endpoint
    server.createContext("/", exchange -> {

        String response = ""
            <html>
                <body>
                    <h1>Hello from Java 18!</h1>
                    <p>Simple HTTP Server</p>
                </body>
            </html>
            "";

        exchange.sendResponseHeaders(
            200,
            response.getBytes().length
        );

        try (OutputStream output =
            exchange.getResponseBody()) {

            output.write(response.getBytes());
        }
    });

    // Start server
    server.start();

    System.out.println(
        "Server started at http://localhost:8000/"
    );
}
}

```

Run

```

javac SimpleWebServer.java
java SimpleWebServer

```

Output:

Server started at http://localhost:8000/

Open:

<http://localhost:8000/>

The browser receives:

Hello from Java 18!
Simple HTTP Server

Module Requirement

If you're using the Java module system, the HTTP server API comes from:

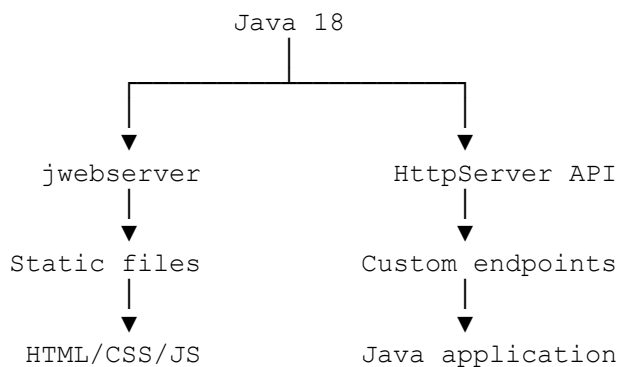
```
module my.application {  
    requires jdk.httpserver;  
}
```

Important Distinction

There are two related ways to use this Java 18 capability:

Approach	Purpose
jwebserver	Quickly serve static files
HttpServer API	Build a small programmatic HTTP server

For example:



Key Characteristics

Feature	Simple Web Server
Introduced	Java 18
Year	2022

Command	jwebserver
Default port	8000
Protocol	HTTP
Primary purpose	Static file serving
External framework	Not required
Production web server	No
Underlying API	jdk.httpserver
Useful for	Development, testing, learning

In short: Java 18's **Simple Web Server** provides a convenient way to start a lightweight HTTP server directly from the JDK, especially for serving static files without installing a separate web server or framework.